

Week 03

Verification Planning and Testbenches

Jesus Sanchez

Dept. d'Arquitectura de Computadors
Universitat Politècnica de Catalunya

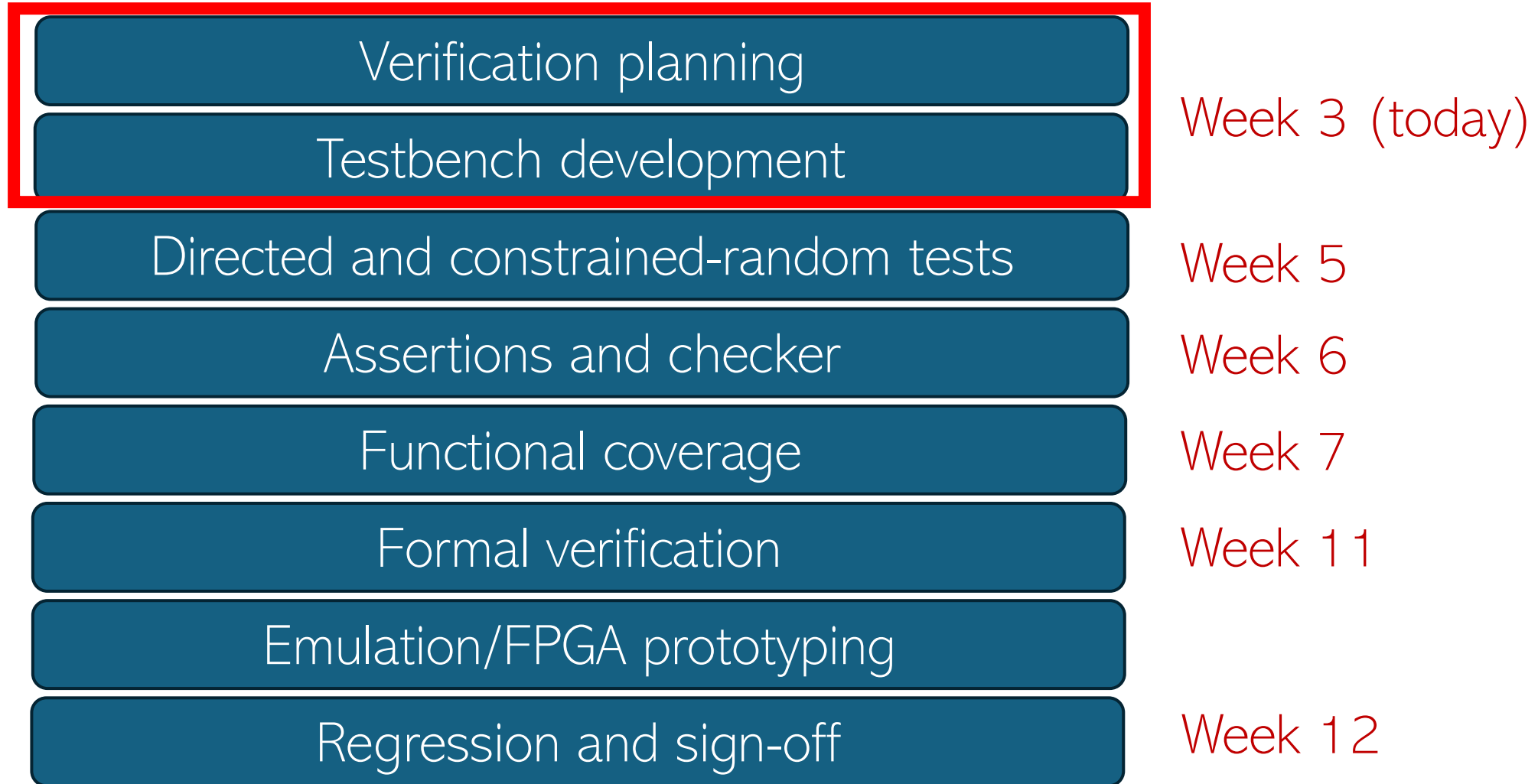
Processor Verification

Master in Innovation and Research in Informatics

Objectives of the Week

- Understand the role of **simulation** in pre-silicon functional verification
- Identify the **components** of a SystemVerilog **testbench**
- Develop a basic **verification plan** based on design requirements
- **Best practices** for writing reusable, scalable, and maintainable testbenches

Verification Flow

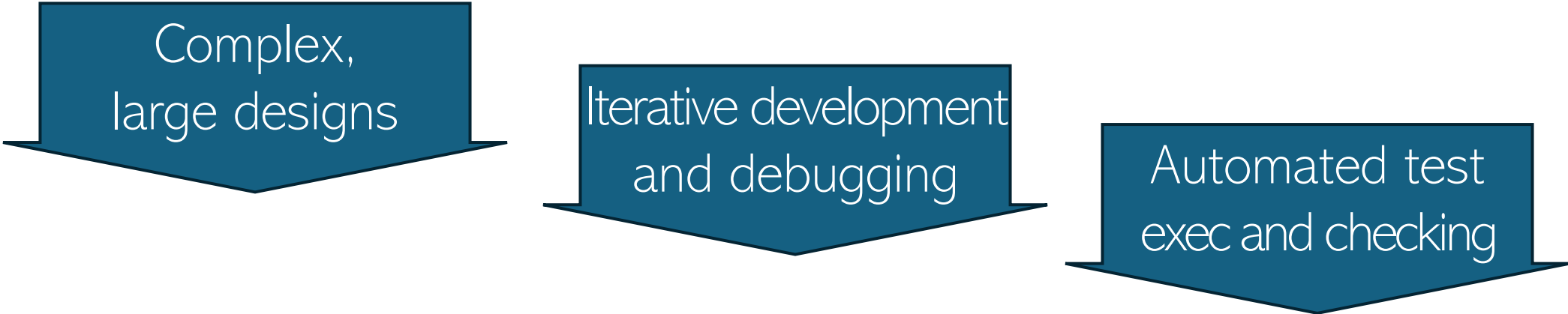


Agenda

1. Foundations of simulation-based verification
2. Verification planning
3. Testbench architecture and components
4. Wrap-up and self-learning

Why Simulation-Based Verification?

- What is Simulation-Based Verification?
 - A dynamic verification method where design models are executed with stimulus inputs to check behavior
- Why Use Simulation?



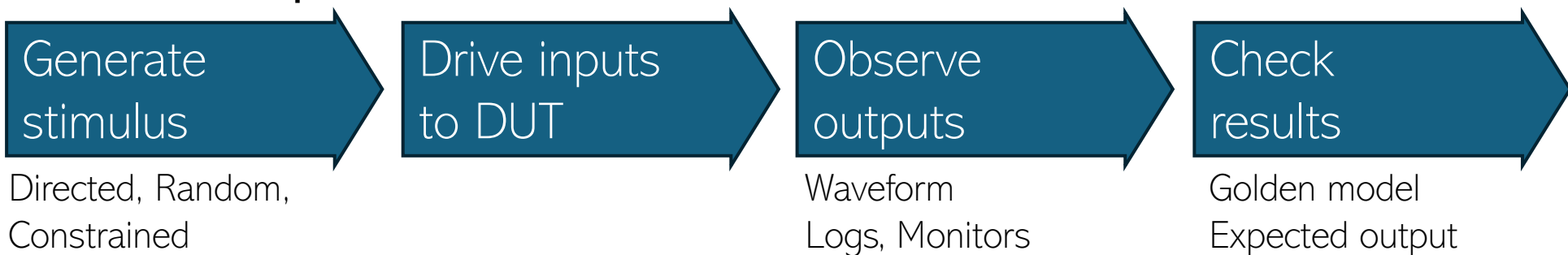
Complex,
large designs

Iterative development
and debugging

Automated test
exec and checking

Stimulus-Response Philosophy

- Verification by simulation is based on a stimulus-response model:
 - “Apply inputs (stimulus), observe outputs (response), and compare to expected behavior.”
- Why This Model?
 - Aligns with black-box testing: treat DUT as a function with inputs and outputs
 - Allows separation of concerns: stimulus generation vs. response checking
- Simulation Loop



Simulation vs Formal Verification

Aspect	Simulation-Based	Formal Verification
Approach	Dynamic: stimulus-based execution	Static: exhaustive mathematical proof
Scope	Explores specific input scenarios	Explores all possible input scenarios
Scalability	Scales well to large designs	Limited to smaller or well-partitioned logic
Debuggability	Visual outputs, waveforms, and logs	Counter-examples for failing properties
Exhaustiveness	Not exhaustive : relies on quality of test cases	Exhaustive within constraints and property bounds
Common Use Cases	Functional testing, system-level scenarios	Protocol checking, safety properties, corner-case checks
Result Type	Pass/Fail per test case	Proof/Counterexample of a property
Tools	ModelSim, Questa, VCS, etc.	JasperGold, VC Formal, etc.

Formal proves correctness $\leftrightarrow$ Simulation builds confidence
A hybrid approach often yields the best coverage and confidence

Simulation vs Emulation

Aspect	Simulation-Based	Emulation
Speed	Slower : software-based execution	Much faster : hardware-accelerated
Scalability	Suitable for small to medium designs	Handles very large SoCs and full systems
Setup Time	Easier and quicker to set up	Requires hardware platform setup
Cost	Lower (runs on general-purpose machines)	Higher (dedicated emulation hardware needed)
Use Case	RTL debug, unit/block testing	System validation, software bring-up
Debug Capabilities	Full waveform and signal visibility	Limited visibility, requires probes and dumps
Turnaround Time	Good for quick iterations	Best for regression and large test suites
Tools	ModelSim, Questa, VCS, etc.	Cadence Palladium, Synopsys ZeBu, Siemens Veloce

Use simulation during early RTL development,
and emulation when full-system validation and software co-development are needed

Agenda

1. Foundations of simulation-based verification
2. Verification planning
3. Testbench architecture and components
4. Wrap-up and self-learning

What is a Verification Plan?

■ Definition

- A **Verification Plan** is a structured document that outlines:
 - 1) What needs to be verified in the design
 - 2) How it will be verified (methods, stimulus types)
 - 3) When it will be verified (milestones, timelines)
 - 4) Who is responsible for each part

■ Purpose

- Bridges the gap between **design specification** and **testbench development**
- Enables **measurable progress tracking**

The verification plan is a contract between the design team and the verification team about what will be verified and how

Contents of a Verification Plan

Minimum Verification Plan (for small designs):

- Design overview (what is the DUT?)
- Features to verify (from spec)
- Test strategy per feature
- Pass/fail criteria

Extended Verification Plan (as design scales):

- Coverage goals (functional/code)
- Testbench architecture
- Assumptions & constraints
- Reuse opportunities
- Tools and regressions

Traceability matrix:

- What is being verified (feature/requirement)
- How it is being verified (test strategy)

Traceability Matrix

■ Why It Matters

- The ability to track each verification activity back to a **specific design requirement** or **specification item**
 - Ensures **complete coverage** of all functional requirements
 - Helps identify **untested** features or **redundant** tests
 - Aids in impact analysis when **specs change**

■ Example

Req ID	Requirement	Feature	Test ID	Coverage Goal
RQ-001	Adder supports carry-in/out	Carry logic	TB_ADD_CARRY	100% functional
RQ-002	Sum is correct for all inputs	Core adder logic	TB_ADD_RESULT	100% toggle
RQ-003	Output valid signal on result	Valid output	TB_ADD_VALID	Assertion + FC

■ Tools that Help

- **Spreadsheet**-based matrices
- Requirements management tools (e.g., Jama, DOORS) – **Project managers**
- Integrated verification planning in **EDA tools** (e.g., Cadence vManager, Synopsys VC Plans)

Testbench vs Test

■ A testbench

- It is the verification infrastructure
- Written once
- Reuse for many tests
- Enables testing, but not a test by itself

■ A test

- It is a specific experiment (or scenario) run using the testbench infra
- Defined by a particular stimulus sequence (aka. one or many **input vectors**)
 - One stimulus = one input vector = apply signals to DUT input interface

Example: Counting test (single test)

Intent:

Verify that the counter increments correctly when enable = 1

What it requires:

- ResetEnable asserted
- Several clock cycles
- Observation of a sequence of outputs



Still one test



Because it verifies one feature

A test is defined by its intent, not by how many vectors it uses

Metrics for Success

■ Why Metrics Matter?

- Help **assess progress**, quality, and confidence in the verification process
- Provide a quantitative basis for "done" decisions
- Enable early identification of gaps or inefficiencies

■ Key Metrics

Metric	Description	Goal
Written/Pass Rate	% of testcases that written/pass successfully	100% at tape-out
Bug Count	Total number of open vs resolved bugs	Zero critical bugs
Assertion Hits/Failure	Tracks whether assertions are triggered and how often	Failures = design/testbench bugs
Code Coverage	Measures how much of the code has been executed	High (>90%) but not a guarantee
Functional Coverage	Measures if all functional scenarios were exercised	90-100% depending on goals

■ Metric Tracking Over Time

- Visualize trends in a dashboard or graph
- Use to guide resource allocation and re-verification focus

Verification Plan Example: 4-bit Binary Counter

Definition Specification Document

Title: Specification for 4-Bit Binary Up-Counter

Module Name: binary_counter_4bit

1. Functional Description

The 4-bit binary counter counts upwards on every rising edge of the clock signal, provided the enable input is asserted. It can be asynchronously reset to zero via the reset input

2. Interface

Signal	Direction	Width	Description
clk	Input	1	Clock signal
reset	Input	1	Asynchronous reset (active high)
enable	Input	1	Enables counting when high
count	Output	4	Current count value (0-15)

3. Behavioral Description:

- On reset = 1, count is immediately reset to 0
- On rising edge of clock, if enable = 1 and reset = 0, the counter increments by 1
- On overflow (from 4'b1111), it wraps around to 4'b0000

Design RTL

File: binary_counter_4bit.v

```
1  module binary_counter_4bit (  
2      input  wire clk,  
3      input  wire reset,  
4      input  wire enable,  
5      output reg [3:0] count  
6  );  
7  
8  always @(posedge clk or posedge reset) begin  
9      if (reset)  
10         count <= 4'b0000;  
11     else if (enable)  
12         count <= count + 1;  
13 end  
14  
15 endmodule
```

Verification Plan Example: 4-bit Binary Counter

1. DUT Overview

- Module: `binary_counter_4bit`
- Function: 4-bit up-counter with clock, reset, and enable inputs

2. Verification Objectives

- Verify correct count sequence from 0 to 15
- Confirm behavior under reset and enable conditions
- Check wraparound from 15 to 0

3. Features to Verify & Test Strategy per Feature

Feature	Description	Test Strategy
Count Sequence	Increments by 1 every clock cycle	Drives 16+ clock cycles with enable=1
Reset Behavior	Reset brings count to 0 immediately	Assert reset at various count values
Enable Control	Not count when enable=0	Toggle enable during count
Wraparound	Rolls over to 0 after 15	Run enough cycles to observe rollover

4. Testbench Architecture

- Simple behavioral reference model to predict expected value
- DUT instantiation and connectivity
- Clock and reset generation logic
- Scoreboard comparison with expected count

5. Coverage Goals

- Functional Coverage:
 - Counter hits all values 0–15
 - Reset activated in different count states
 - Enable toggled mid-count
- Code Coverage: Aim for 100% line/branch coverage (should be achievable)

6. Pass/Fail Criteria

- Counter value always matches expected
- Reset always sets to 0 immediately
- No incorrect transitions or timing violations

10:00

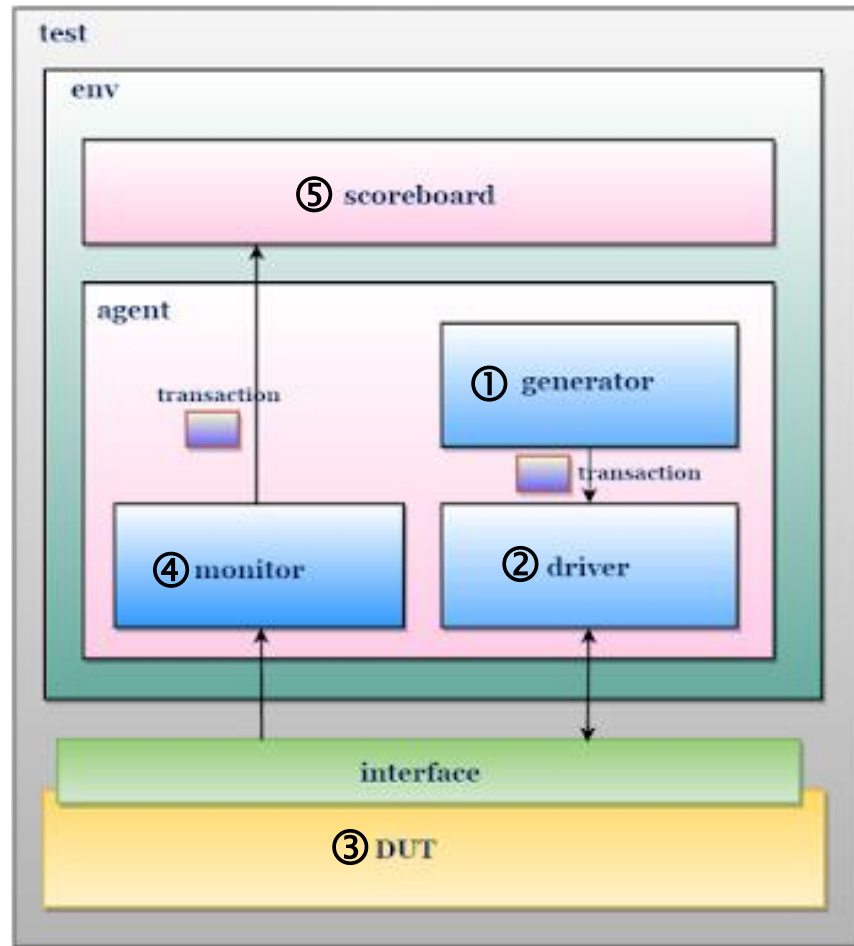
Agenda

1. Foundations of simulation-based verification
2. Verification planning
3. Testbench architecture and components
4. Wrap-up and self-learning

What Makes a Good Techbench?

- **Self-checking**
 - Automatically verifies DUT (Design Under Test) outputs without manual inspection
- **Repeatable and deterministic**
 - Produces the same result every time, regardless of system or run
- **Readable and modular**
 - Easy to understand and modify, uses modular components (e.g., drivers, monitors)
- **Reusable**
 - Can be adapted for new features or other designs with minimal change
- **Scalable**
 - Works for simple modules and larger SoCs with minimal refactor
- **Automated**
 - Includes automation for generation, simulation, and result checking

Basic Testbench Architecture



- ① **Generator:**
 - Creates test vectors (manual or randomized)
- ② **Driver:**
 - Applies the vectors to the DUT through interface signals
- ③ **DUT (Device Under Test):**
 - The design being verified
- ④ **Monitor:**
 - Passively observes DUT outputs
- ⑤ **Scoreboard:**
 - Compares DUT output to expected behavior
 - Tracks overall test results and data consistency

Source:

[SystemVerilog TestBench - Verification Guide](#)

DUT Instantiation and Signal Connectivity

```
1 module binary_counter_4bit (  
2     input  wire clk,  
3     input  wire reset,  
4     input  wire enable,  
5     output reg [3:0] count  
6 );  
7  
8 always @(posedge clk or posedge reset) begin  
9     if (reset)  
10        count <= 4'b0000;  
11     else if (enable)  
12        count <= count + 1;  
13 end  
14  
15 endmodule
```



```
1 module counter_tb;  
2  
3     // Signal declarations  
4     logic _clk, _reset, _enable;  
5     logic [3:0] _count;  
6  
7     // Instantiate DUT  
8     binary_counter_4bit dut (  
9         .clk(_clk),  
10        .reset(_reset),  
11        .enable(_enable),  
12        .count(_count)  
13    );  
14  
15    ...
```

Named Port Connections

- Order does not matter
- Recommended for larger designs

```
1 module counter_tb;  
2  
3     // Signal declarations  
4     logic _clk, _reset, _enable;  
5     logic [3:0] _count;  
6  
7     // Instantiate DUT  
8     binary_counter_4bit dut (  
9         _clk,  
10        _reset,  
11        _enable,  
12        _count  
13    );  
14  
15    ...
```

Positional Port Connections

- Order matters
- Harder to maintain and review
- Less common today

Generators: Purpose and Patterns

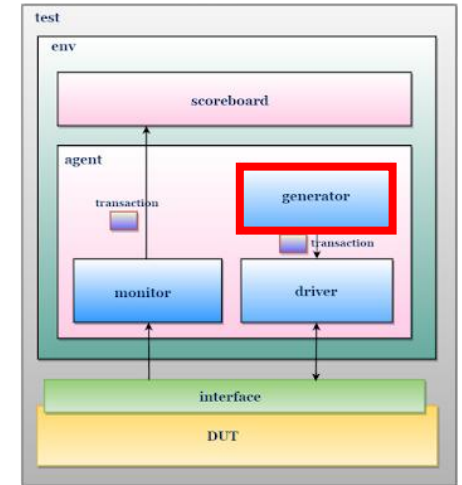
■ Purpose of Generators

- Automate stimulus generation
- Enable scalable and repeatable testing
- Support directed and random stimulus
- Help achieve coverage goals

■ Typical Generator Responsibilities

- Create legal transactions or signal sequences
- Apply constraints (e.g., protocol correctness, value ranges)
- Track previously generated values (for randomness control)

Pattern	Description	Use Case Example
Linear/Sequential	Generates predictable, ordered values	Address sweeping, regressions
Random	Fully random values	Fuzzing, corner-case exploration
Constrained Random	Random values obeying constraints	Coverage-drive verification
State-Aware	Considers DUT state to generate inputs	Protocols, handshaking logic



Generator builds transactions → Driver converts them to signal-level activity

Generator: an Example

binary_counter_4bit.v

```
1 module binary_counter_4bit (
2     input wire clk,
3     input wire reset,
4     input wire enable,
5     output reg [3:0] count
6 );
7
8 always @(posedge clk or posedge reset) begin
9     if (reset)
10        count <= 4'b0000;
11    else if (enable)
12        count <= count + 1;
13 end
14
15 endmodule
```

counter_generator.sv

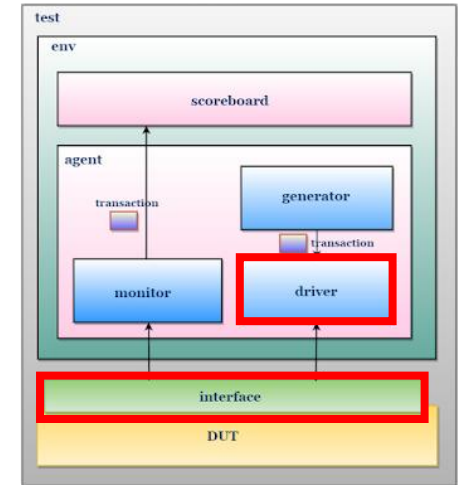
```
1 class counter_generator; // Transaction definition
2     rand bit reset;
3     rand bit enable;
4
5     function void randomize_inputs();
6         this.randomize(); // Generates random values
7     endfunction
8 endclass
```

binary_counter_4bit_tb.sv

```
1 timescale 1ns/1ps
2 include "counter_generator.sv"
3
4 module binary_counter_4bit_tb;
5
6     // Signal declarations
7     logic clk, reset, enable;
8     logic [3:0] count;
9
10    // Clock Generation: 10ns period
11    initial clk = 0;
12    always #5 clk = ~clk;
13
14    // Instantiate DUT
15    binary_counter_4bit dut (
16        .clk(clk),
17        .reset(reset),
18        .enable(enable),
19        .count(count)
20    );
21
22    // Create generator object
23    counter_generator gen;
24
25    initial begin
26        // Reset sequence
27        reset = 1;
28        enable = 0;
29        #5 reset = 0;
30
31        gen = new();
32
33        repeat (10) begin
34            gen.randomize_inputs();
35            reset = gen.reset;
36            enable = gen.enable;
37            #10; // wait for a clock cycle
38        end
39
40        $display("Test completed");
41        $stop;
42    end
43
44 endmodule
```

Drivers and Interfaces Responsibilities

- A driver is responsible for
 - Receiving transactions (e.g., from a generator)
 - Driving DUT inputs accordingly, often with timing control
 - Abstracting away signal details from the test sequence
- An interface in a container for signals
 - It encapsulates the connection between modules e.g., testbench ↔ DUT
 - Advantages:
 - Reduces repetition: Instead of declaring and connecting each signal individually
 - Improves readability: Clarifies signal intent and grouping — helps document the design



Generator builds transactions → Driver converts them to signal-level activity

Driver and Interface: Example

counter_if.sv

```
1 interface counter_if(input bit clk);
2     logic reset;
3     logic enable;
4     logic [3:0] count;
5 endinterface
```

counter_driver.sv

```
1 `include "counter_if.sv"
2
3 class counter_driver;
4     virtual interface counter_if vif;
5
6     function new(virtual interface counter_if vif);
7         this.vif = vif;
8     endfunction
9
10    task apply(input bit reset_val, input bit enable_val);
11        @(posedge vif.clk);
12        vif.reset <= reset_val;
13        vif.enable <= enable_val;
14    endtask
15 endclass
```

binary_counter_4bit_tb.sv

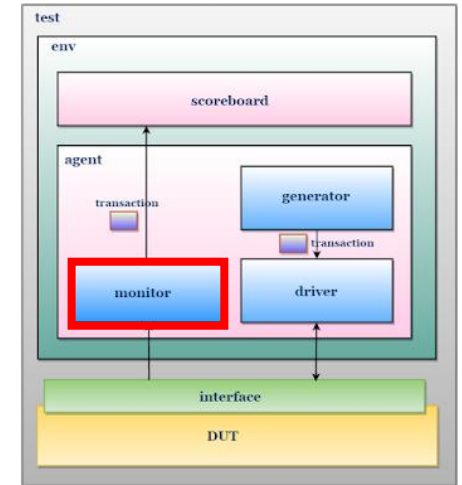
```
1 `timescale 1ns/1ps
2 `include "counter_generator.sv"
3 `include "counter_driver.sv"
4 `include "counter_if.sv"
5
6 module binary_counter_4bit_tb;
7
8     // Signal declarations
9     logic clk;
10
11    // Clock Generation: 10ns period
12    initial clk = 0;
13    always #5 clk = ~clk;
14
15    counter_if cif(clk);
16
17    // Instantiate DUT
18    binary_counter_4bit dut (
19        .clk(clk),
20        .reset(cif.reset),
21        .enable(cif.enable),
22        .count(cif.count)
23    );
24
```

```
25 // Create generator and driver objects
26 counter_generator gen;
27 counter_driver drv;
28
29 initial begin
30     // Reset sequence
31     cif.reset = 1;
32     cif.enable = 0;
33     #5 cif.reset = 0;
34
35     gen = new();
36     drv = new(cif);
37
38     repeat (10) begin
39         gen.randomize_inputs();
40         drv.apply(gen.reset, gen.enable);
41     end
42
43     $display("Test completed");
44     $stop;
45 end
46
47 endmodule
```

Monitors and Response Capture

■ What Is a Monitor?

- A monitor is a passive testbench component that observes signals on an interface or DUT without influencing them. It extracts relevant information (trx) and often sends it to:
 - Scoreboards for comparison
 - Coverage collectors
 - Logs or transcript reports



■ Key Responsibilities:

Role	Description
Passive Observation	Reads signals or interface values; does not drive them
Transaction Extraction	Converts signal activity into high-level transactions
Response Capture	Records DUT outputs for later checking
Synchronization	Can wait on clocks or handshakes to align with DUT behavior

Monitor: an Example

counter_monitor.sv

```
1  `include "counter_if.sv"
2
3  class counter_monitor;
4      virtual interface counter_if vif;
5
6      function new(virtual interface counter_if vif);
7          this.vif = vif;
8      endfunction
9
10     task monitor_signals();
11         forever begin
12             @(posedge vif.clk);
13             $display("Time: %0t | Reset: %0b | Enable: %0b | "
14                     + "Count: %0d",
15                     $time, vif.reset, vif.enable, vif.count);
16         end
17     endtask
18 endclass
```

binary_counter_4bit_tb.sv

```
1  `timescale 1ns/1ps
2  `include "counter_generator.sv"
3  `include "counter_driver.sv"
4  `include "counter_if.sv"
5  `include "counter_monitor.sv"
6
7  module binary_counter_4bit_tb;
8
9      // Signal declarations
10     logic clk;
11
12     // Clock Generation: 10ns period
13     initial clk = 0;
14     always #5 clk = ~clk;
15
16     counter_if cif(clk);
17
18     // Instantiate DUT
19     binary_counter_4bit dut (
20         .clk(clk),
21         .reset(cif.reset),
22         .enable(cif.enable),
23         .count(cif.count)
24     );
25
```

```
26     // Create generator, driver and monitor objects
27     counter_generator gen;
28     counter_driver drv;
29     counter_monitor mon;
30
31     initial begin
32         // Reset sequence
33         cif.reset = 1;
34         cif.enable = 0;
35         #5 cif.reset = 0;
36
37         gen = new();
38         drv = new(cif);
39         mon = new(cif);
40
41         // Start monitor task in parallel
42         fork
43             | mon.monitor_signals();
44         join_none
45
46         repeat (10) begin
47             gen.randomize_inputs();
48             drv.apply(gen.reset, gen.enable);
49         end
50
51         $display("Test completed");
52         $stop;
53     end
```

Checkers vs Scoreboard

- What is a Checker?

- Compares DUT output against expected values
- Detects mismatches (bugs) and reports them automatically

- Implementation Styles:

Style	Description
Inline	Assertions or conditions inside the testbench
Checker Module	Separate SV module to encapsulate checking
Scoreboard-Based	Compare DUT output to a reference model

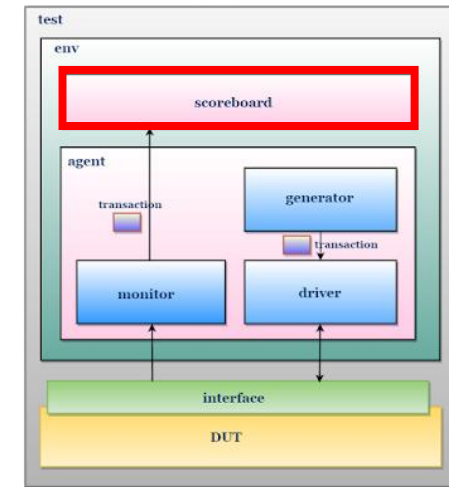
A **checker** might validate: “The count signal must increase only if enable=1.”

A **scoreboard** might compare: “The actual count at time N must match the expected result from the reference model.”

- What is a Scoreboard?

- Collects outputs from both the DUT and the reference model
- Compares them to verify correctness
- Logs mismatches or confirms correctness.

High-level, golden implementation



Updated Monitor with Transactions

- What a Transaction?
 - A transaction is defined as a class that bundles a set of related signal values into a single object.
 - Think of it as a "packet" or "operation" rather than individual wires.

counter_transaction.sv

```
1 class counter_transaction;
2     rand logic reset;
3     rand logic enable;
4     logic [3:0] count; // Observed value after clock
5
6     function void display();
7         $display("reset=%0b, enable=%0b, count=%0d", reset, enable, count);
8     endfunction
9 endclass
```

counter_monitor.sv

```
1 class counter_monitor;
2     virtual interface counter_if vif;
3     mailbox #(counter_transaction) mon2chk;
4     mailbox #(counter_transaction) mon2scb;
5
6     function new(virtual interface counter_if vif,
7                 mailbox #(counter_transaction) mon2chk,
8                 mailbox #(counter_transaction) mon2scb);
9         this.vif = vif;
10        this.mon2chk = mon2chk;
11        this.mon2scb = mon2scb;
12    endfunction
13
14    task monitor_signals();
15        counter_transaction trx;
16        forever begin
17            @(posedge vif.clk);
18            trx = new();
19            trx.reset = vif.reset;
20            trx.enable = vif.enable;
21            trx.count = vif.count;
22            mon2chk.put(trx);
23            mon2scb.put(trx);
24        end
25    endtask
26 endclass
```

Checker: an Example

counter_checker.sv

```
1  class counter_checker;
2      mailbox #(counter_transaction) mon2chk;
3
4      function new(mailbox #(counter_transaction) mon2chk);
5          this.mon2chk = mon2chk;
6      endfunction
7
8      task run();
9          counter_transaction prev_trx, curr_trx;
10
11         mon2chk.get(prev_trx); // first cycle
12
13         forever begin
14             mon2chk.get(curr_trx);
15             if (!curr_trx.reset && curr_trx.enable) begin
16                 assert (curr_trx.count == prev_trx.count + 1)
17                 else $error("Checker: count did not increment correctly!");
18             end
19             prev_trx = curr_trx;
20         end
21     endtask
22 endclass
```

Scoreboard: Example

counter_refmodel.sv

```
1  class counter_refmodel;
2      logic [3:0] ref_count;
3
4      function new();
5          ref_count = 0;
6      endfunction
7
8      function void reset();
9          ref_count = 0;
10     endfunction
11
12     function void update(logic reset, logic enable);
13         if (reset)
14             ref_count = 0;
15         else if (enable)
16             ref_count++;
17     endfunction
18
19     function logic [3:0] get_count();
20         return ref_count;
21     endfunction
22 endclass
```

counter_scoreboard.sv

```
1  class counter_scoreboard;
2      mailbox #(counter_transaction) mon2scb;
3      counter_ref_model refmodel;
4
5      function new(mailbox #(counter_transaction) mon2scb);
6          this.mon2scb = mon2scb;
7          this.ref_model = new();
8      endfunction
9
10     task run();
11         counter_transaction trx;
12
13         forever begin
14             mon2scb.get(trx);
15
16             // Update golden reference model
17             refmodel.update(trx.reset, trx.enable);
18
19             // Compare DUT output with reference
20             if (trx.count != refmodel.get_count()) begin
21                 $error("Scoreboard mismatch: expected %0d, got %0d",
22                     refmodel.get_count(), trx.count);
23             end
24         end
25     endtask
26 endclass
```

Scoreboard: Refmodel Integrated or Separated

```
1 class counter_scoreboard;
2   mailbox #(counter_transaction) mon2scb;
3   logic [3:0] ref_count;
4
5   function new(mailbox #(counter_transaction) mon2scb);
6     this.mon2scb = mon2scb;
7     ref_count = 0;
8   endfunction
9
10  task run();
11    counter_transaction trx;
12
13    forever begin
14      mon2scb.get(trx);
15
16      if (trx.reset)
17        ref_count = 0;
18      else if (trx.enable)
19        ref_count++;
20
21      if (trx.count != ref_count)
22        $error("Scoreboard mismatch: expected %0d, got %0d",
23              ref_count, trx.count);
24    end
25  endtask
26 endclass
```

```
1 class counter_scoreboard;
2   mailbox #(counter_transaction) mon2scb;
3   counter_ref_model refmodel;
4
5   function new(mailbox #(counter_transaction) mon2scb);
6     this.mon2scb = mon2scb;
7     this.ref_model = new();
8   endfunction
9
10  task run();
11    counter_transaction trx;
12
13    forever begin
14      mon2scb.get(trx);
15
16      // Update golden reference model
17      refmodel.update(trx.reset, trx.enable);
18
19      // Compare DUT output with reference
20      if (trx.count != refmodel.get_count()) begin
21        $error("Scoreboard mismatch: expected %0d, got %0d",
22              refmodel.get_count(), trx.count);
23      end
24    end
25  endtask
26 endclass
```


Complete Testbench: an Example

binary_counter_4bit_tb.sv

```
1  `timescale 1ns/1ps
2  `include "counter_generator.sv"
3  `include "counter_driver.sv"
4  `include "counter_if.sv"
5  `include "counter_monitor.sv"
6  `include "counter_transaction.sv"
7  `include "counter_checker.sv"
8  `include "counter_scoreboard.sv"
9
10 module binary_counter_4bit_tb;
11
12     // Signal declarations
13     logic clk;
14
15     // Clock Generation: 10ns period
16     initial clk = 0;
17     always #5 clk = ~clk;
18
19     counter_if cif(clk);
20
21     // Instantiate DUT
22     binary_counter_4bit dut (
23         .clk(clk),
24         .reset(cif.reset),
25         .enable(cif.enable),
26         .count(cif.count)
27     );
```

```
28
29     // Create component objects
30     counter_generator gen;
31     counter_driver drv;
32     counter_monitor mon;
33     counter_checker chk;
34     counter_scoreboard scb;
35
36     // Mailboxes
37     mailbox #(counter_transaction) mbox_chk = new();
38     mailbox #(counter_transaction) mbox_scb = new();
39
40     initial begin
41         // Reset sequence
42         cif.reset = 1;
43         cif.enable = 0;
44         #5 cif.reset = 0;
45
46         gen = new();
47         drv = new(cif);
48         mon = new(cif, mbox_chk, mbox_scb);
49         chk = new(mbox_chk);
50         scb = new(mbox_scb);
51
```

```
52     // Start monitor and checker tasks in parallel
53     fork
54         mon.monitor_signals();
55         chk.run();
56         scb.run();
57     join_none
58
59     repeat (10) begin
60         gen.randomize_inputs();
61         drv.apply(gen.reset, gen.enable);
62     end
63
64     $display("Test completed");
65     $stop;
66 end
67
68 endmodule
```

Agenda

1. Foundations of simulation-based verification
2. Verification planning
3. Testbench architecture and components
4. Wrap-up and self-learning

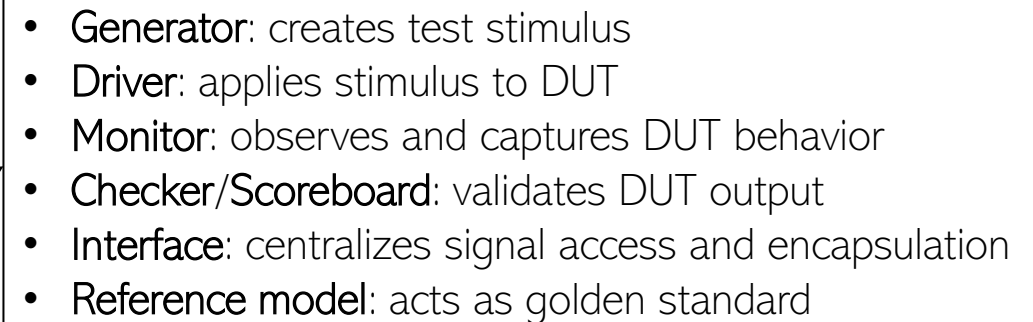
Summary of Key Concepts from this Session

■ Foundations of Simulation-Based Verification

- Purpose: detect design bugs before silicon
- Relies on testbenches, stimuli, and monitors

■ Verification Planning

- Based on design specification and DUT analysis.
- Defines:
 - Features to verify
 - Test strategies per feature
 - Coverage goals (functional/code)

- 
- **Generator:** creates test stimulus
 - **Driver:** applies stimulus to DUT
 - **Monitor:** observes and captures DUT behavior
 - **Checker/Scoreboard:** validates DUT output
 - **Interface:** centralizes signal access and encapsulation
 - **Reference model:** acts as golden standard

■ Testbench Architecture and Components

Link to Next Week

- **Week 04: SystemVerilog for Verification**
 - **SystemVerilog Classes**: building blocks for reusable components
 - **Data Types**: logic vs reg vs wire
 - **Randomization Techniques**: rand, randc, constraints
 - **Interfaces and Modports**: organizing complex connections
 - Using **Mailboxes** and **Semaphores** for component communication
 - Building reusable **verification components** (UVCs)

Self-Learning Corner

- Books

- Chapter 1: Introduction to Verification (Spear)

- Presentations

- [Proven Strategies for Better Verification Planning](#)

- Articles

- [The Art of Crafting an Unbeatable Verification Plan](#)

- Practice

- EDA Playground: Play with the 4-bit counter

“If you think good design is expensive, you should look at the cost of bad design”

Ralph Speth, former Jaguar CEO

